



نام درس: شبکه های کامپیوتری
نویسنده: محمد امانلو - ۸۱۰۱۰۰۰۸۴



تکلیف شماره ۲

سوال ۱

الف) نسخه به کاررفته

در خط وضعیت آمده است: HTTP/1.1 200 OK. فیلد اول این خط نسخه پروتکل را مشخص می کند؛ پس نسخه HTTP/1.1 است.

ب) موفقیت ارسال

کد وضعیت 200 OK نشان می دهد درخواست موفق بوده و شیء در پاسخ برگردانده می شود؛ بنابراین ارسال موفق است.

ج) اندازه سند

Header Content-Length تعداد بایت های شیء ارسالی را می دهد. مقدار درج شده ۹۹۳،۵۳ بایت است.

د) نوع فایل ارسالی

نوع رسانه ای بدنه پاسخ با Header Content-Type تعیین می شود. در این پیام مقدار image/html گزارش شده است؛ پس نوع محتوا همین است.

در HTTP/1.1 اتصال پایدار حالت پیش فرض است. Headerهای Connection: keep-alive و Keep-Alive: time-out=43, max=70 نشان می دهند سرور اتصال TCP را برای چند درخواست/پاسخ باز نگه می دارد.

سوال ۲

منطق: در این معماری فقط سرور می فرستد؛ پس زمان نهایی باید هم از سمت سرور قابل تحقق باشد و هم از سمت کندترین گیرنده. دو کران داریم: ارسال کل NF با نرخ u_s و دریافت یک نسخه با نرخ $d_{\min}$. در نتیجه فرمول کلی حداقل زمان:

$$D_{cs} = \max \left\{ \frac{NF}{u_s}, \frac{F}{d_{\min}} \right\}$$

محاسبه:

$$\frac{F}{d_{\min}} = \frac{5 \times 10^9}{10 \times 10^6} = 500 \text{ s}, \quad \frac{NF}{u_s} = \frac{8 \times 5 \times 10^9}{87 \times 10^6} \approx 459,8 \text{ s}.$$

پس

$$D_{cs} = \max(500, 459,8) = 500 \text{ s}$$

تفسیر: گلوگاه download کندترین peer است؛ حتی با توان upload سرور، دریافت سریع تر از $F/d_{\min}$ ممکن نیست و بنابراین زمان نهایی 500 s به دست می آید.

سوال ۵

درخواست های Iterative

- ۱) بین مراحل ۱ و ۲: ابتدا کش خودش را نگاه می کند Local DNS (اگر نبود، به Root DNS می پرسد).
- ۲) بین مراحل ۲ و ۳: اگر Root DNS آدرس را نداشته باشد، ارجاع به TLD DNS می دهد.
- ۳) بین مراحل ۴ و ۵: اگر TLD DNS آدرس را نداشته باشد، ارجاع به Authoritative DNS.
- ۴) بین مراحل ۶ و ۷: سرور معتبر با رکورد نوع A (در صورت IPv6: AAAA) پاسخ می دهد.

درخواست های Recursive

- ۱) بین مراحل ۱ و ۲: ابتدا کش خودش را نگاه می کند Local DNS؛ در صورت نبودن نتیجه، یک پرس و جوی recursive ارسال می کند.
- ۲) بین مراحل ۲ و ۳: Root DNS درخواست را به TLD DNS ارسال می کند.
- ۳) بین مراحل ۴ و ۵: Authoritative DNS پاسخ را به همان سروری که از او پرسیده بود برمی گرداند؛ در این زنجیره یعنی به TLD DNS.
- ۴) بین مراحل ۶ تا ۸: پاسخ در مسیر معکوس تا Local DNS و سپس کاربر بازمی گردد؛ نوع رکورد A (یا AAAA) است.

از دید مقیاس پذیری شبکه، روش iterative بار را بین سرورها توزیع می کند و مناسب تر است؛ recursive برای کاربر ساده تر است ولی بار بیش تری روی سرورهای بالادست می گذارد. (در عمل، ترکیب رایج: کاربر Local DNS به صورت recursive، و بقیه مسیر iterative)

سوال ۷

تعریف متغیرها.

 d_{lan} : زمان متوسطی که هر فایل داخل LAN (شامل صف/انتقال روی لینک دسترسی) می گذراند. d_{net} : زمان متوسط بیرون از LAN (مسیر اینترنت تا سرور مبدأ).نرخ اصابت cache: $p = 0,4$

$$\underbrace{d_{lan} + d_{net}}_{\text{بدون cache}} = 2s \implies d_{net} = 2 - d_{lan}.$$

با فعال بودن cache، تأخیر متوسط بر اساس قانون امید ریاضی مخلوط به صورت زیر است:

$$\mathbb{E}[T] = p d_{lan} + (1 - p)(d_{lan} + d_{net}) = 1,7s.$$

$$p d_{lan} + (1 - p)(d_{lan} + (2 - d_{lan})) = 1,7 \implies d_{lan} + (1 - p) d_{net} = 1,7,$$

$$d_{lan} + 0,6(2 - d_{lan}) = 1,7 \implies 0,4 d_{lan} = 0,5 \implies \boxed{d_{lan} = 1,25s}.$$

$$d_{net} = 2 - 1,25 = 0,75s.$$

پس به طور متوسط هر فایل ۱,۲۵ ثانیه در LAN صرف می کند. cache با اصابت ۴۰٪ باعث می شود بخشی از ترافیک از اینترنت برگردد و میانگین کل از ۲ به ۱,۷ ثانیه کاهش یابد؛ با این حال گلوگاه می تواند لینک دسترسی باشد، بنابراین سهم LAN از تأخیر کل بزرگتر از سهم بیرون از LAN به دست آمده است.

سوال ۹

فرض رایج تحلیل: برای هر آبجکت HTTP در حالت non-persistent یک هزینه 2 RTT (برقراری اتصال + درخواست/اولین بایت) و در حالت persistent به ترتیب بدون pipelining: 1 RTT برای هر آبجکت پس از سند، و با pipelining: یک RTT برای کل دسته درخواست ها داریم. زمان ارسال هر آبجکت = $\frac{\text{اندازه (بیت)}}{\text{نرخ (بیت/ثانیه)}}$.

الف) non-persistent HTTP با اتصالات سری

ابتدا سند اصلی دریافت می‌شود؛ این کار مستلزم $2 \times RTT$ برای تبادلات کنترلی به علاوه زمان انتقال خود محتواس. سپس پنج تصویر به صورت پشت سرهم دریافت می‌گردد که برای هر تصویر نیز همان هزینه $2 \times RTT$ به اضافه زمان انتقال داده پرداخت می‌شود. بنابراین زمان کل برابر است با

$$T = (2 RTT + t_{doc}) + \sum_{i=1}^5 (2 RTT + t_{img,i}) = 6 \cdot 2 RTT + t_{doc} + 5 t_{img},$$

که در آن t_{doc} زمان انتقال سند و t_{img} زمان انتقال هر تصویر است.
زمان کل = زمان سند + جمع زمان ۵ تصویر:

$$T_a = 6 \cdot 2 \cdot 10^{-7} + \frac{8 \times 10^3}{10^6} + 5 \cdot \frac{8 \times 50 \times 10^3}{10^6} = \boxed{2,008,0012 \text{ s}}.$$

ب) non-persistent HTTP با دو اتصال موازی

در این سناریو نیز ابتدا سند اصلی دریافت می‌شود (همان هزینه $2 \times RTT + t_{doc}$). برای تصاویر اما دو connection به صورت parallel فعال‌اند؛ پس زمان نهایی به جای جمع، برابر بیشینه زمان صرف شده روی هر connection است. یکی از اتصالات دو تصویر می‌گیرد و دیگری سه تصویر؛ بنابراین شاخه سه تایی گلوگاه می‌شود. چون برای هر تصویر همان هزینه $2 \times RTT + t_{img}$ را می‌پردازیم، کل زمان برابر است با

$$T = (2 RTT + t_{doc}) + \max\{2(2 RTT + t_{img}), 3(2 RTT + t_{img})\} = (2 RTT + t_{doc}) + 3(2 RTT + t_{img}).$$

زمان کل = زمان سند + max از دو اتصال موازی (یکی ۳ تصویر، دیگری ۲ تصویر):

$$T_b = 2 \cdot 10^{-7} + \frac{8 \times 10^3}{10^6} + 3 \cdot \left(2 \cdot 10^{-7} + \frac{8 \times 50 \times 10^3}{10^6} \right) = \boxed{1,208,0008 \text{ s}}.$$

ج) persistent HTTP

بدون Pipelining. در این حالت یک RTT برای برقراری connection لازم است و سپس یک RTT برای ارسال GET و دیدن اولین بایت سند؛ بعد از آن زمان انتقال خود سند سپری می‌شود. چون اتصال Persistent است، برای تصاویر دیگر نیازی به راه اندازی مجدد اتصال نداریم و هر تصویر تنها یک RTT کنترلی به علاوه زمان ارسال داده می‌طلبد. بنابراین زمان کل برابر است با

$$T_{no-pipe} = (2 RTT + t_{doc}) + 5(RTT + t_{img}).$$

$$T_{c1} = 2 \cdot 10^{-7} + \frac{8 \times 10^3}{1.6} + 5 \cdot \left(10^{-7} + \frac{8 \times 50 \times 10^3}{1.6} \right) = \boxed{2,008,000.7_s}.$$

با **Pipelining**. پس از دریافت سند (مانند قبل)، مرورگر هر پنج مرجع تصویر را یکجا و پیاپی روی همان persistent connection ارسال می کند؛ این دسته درخواست تنها یک RTT هزینه کنترلی دارد و سپس صرفاً انتقال داده های تصاویر انجام می شود. بنابراین کل زمان برابر است با

$$T_{pipe} = (2 RTT + t_{doc}) + RTT + 5 t_{img}.$$

$$T_{c2} = 2 \cdot 10^{-7} + \frac{8 \times 10^3}{1.6} + 10^{-7} + 5 \cdot \frac{8 \times 50 \times 10^3}{1.6} = \boxed{2,008,000.3_s}.$$

۱ سوال ۱۰ آزمایش وب سرور

در این آزمایش مبانی برنامه نویسی سوکت برای اتصالات TCP در پایتون، نحوه ایجاد سوکت، bind به آدرس و پورت مشخص، ارسال و دریافت پیام HTTP و ساخت هدرهای استاندارد تمرین شد. وب سروری پیاده سازی شد که در هر لحظه یک درخواست HTTP را پردازش می کند: اگر فایل درخواستی وجود داشته باشد پاسخ 200 OK به همراه محتوای فایل ارسال می شود و در غیر این صورت پاسخ 404 Not Found بازگردانده می شود.

۱.۱ محیط اجرا و پیکربندی

□ سیستم عامل: Windows 10/11

□ نسخه Python: 3.x

□ آدرس IP سرور (اینترفیس 3 Ethernet): 10.182.63.36

□ پورت گوش دادن: 6789

□ ساختار پوشه: فایل های WebServer.py و HelloWorld.html در یک دایرکتوری واحد (دایرکتوری سرور)

۲.۱ کد وب سرور

کد زیر نسخه کامل شده اسکلت تمرین است. این برنامه با accept اتصال را می پذیرد، خط اول درخواست HTTP را پارس می کند، در صورت وجود فایل هدر 200 و بدنه فایل را می فرستد و در غیر این صورت هدر و بدنه ساده 404 را ارسال می کند.

```
# import socket module
from socket import *
import sys # In order to terminate the program

serverSocket = socket(AF_INET, SOCK_STREAM)

# Prepare a server socket
serverPort = 6789 # listen port
serverSocket.bind(('', serverPort)) # bind to all interfaces
serverSocket.listen(1) # backlog = 1

while True:
    # Establish the connection
    print('Ready to serve...')
    connectionSocket, addr = serverSocket.accept()

    try:
        message = connectionSocket.recv(1024).decode() # receive HTTP
        request
        filename = message.split()[1] # path after GET
        f = open(filename[1:], 'rb') # strip leading "/"

        outputdata = f.read()
        f.close()

        # send HTTP 200 header
        header = (
            "HTTP/1.1 200 OK\r\n"
            "Content-Type: text/html; charset=utf-8\r\n"
            f"Content-Length: {len(outputdata)}\r\n"
            "Connection: close\r\n"
        )
```

```
        "\r\n"
    )
    connectionSocket.send(header.encode())

    # send file body (byte-by-byte per the skeleton)
    for i in range(0, len(outputdata)):
        connectionSocket.send(outputdata[i:i+1])
    connectionSocket.send("\r\n".encode())

    connectionSocket.close()

except IOError:
    # file not found -> send 404
    not_found_body = (
        b"<html><head><title>404 Not Found</title></head>"
        b"<body><h1>404 Not Found</h1><p>File not found.</p></body></"
        b"html>"
    )
    header = (
        "HTTP/1.1 404 Not Found\r\n"
        "Content-Type: text/html; charset=utf-8\r\n"
        f"Content-Length: {len(not_found_body)}\r\n"
        "Connection: close\r\n"
        "\r\n"
    )
    connectionSocket.send(header.encode())
    connectionSocket.send(not_found_body)
    connectionSocket.close()

serverSocket.close()
sys.exit() # Terminate the program after sending the corresponding data
```

۳.۱ فایل آزمون HTML

فایل ساده HelloWorld.html در همان دایرکتوری سرور قرار گرفت:

```
<!doctype html>  
<html><body><h1>Hello from my socket server!</h1></body></html>
```

۴.۱ نحوه اجرا و آدرس های تست

۱. اجرای سرور:

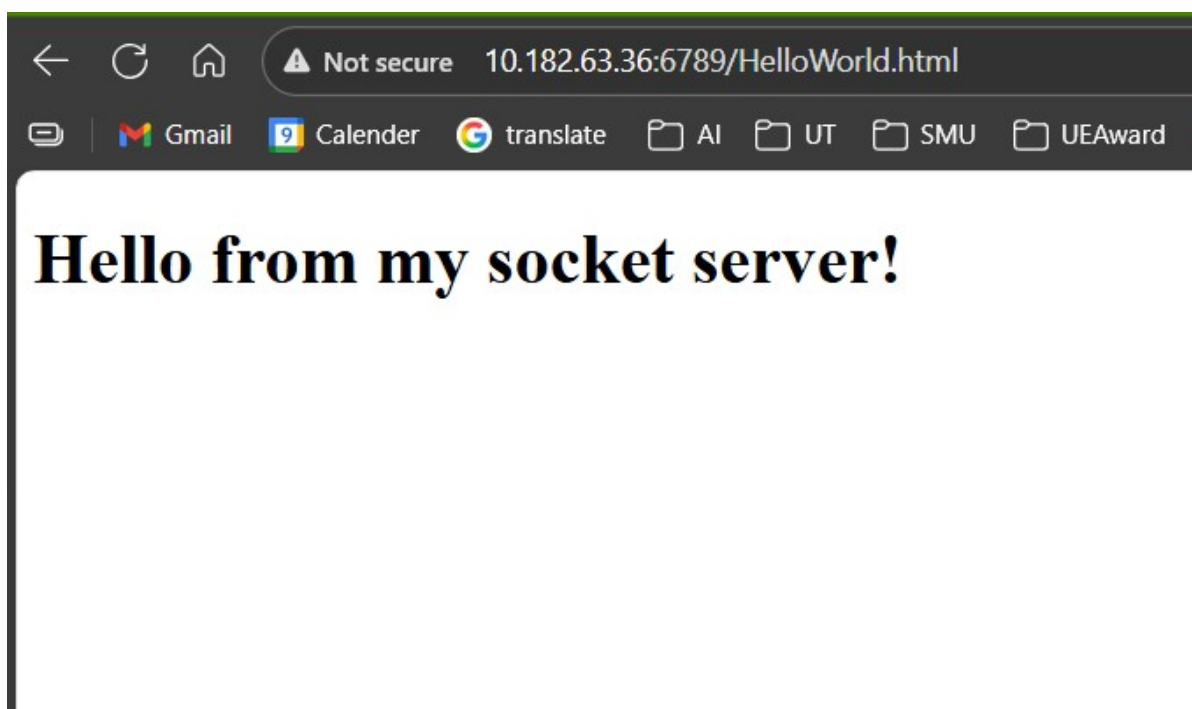
```
python WebServer.py
```

۲. تست موفق روی localhost: http://127.0.0.1:6789/HelloWorld.html

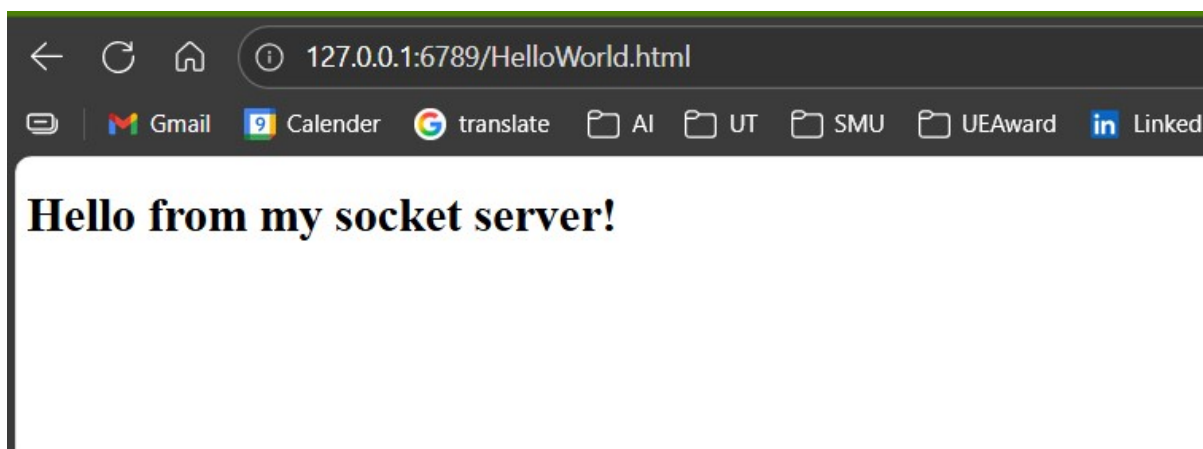
۳. تست موفق روی LAN: http://10.182.63.36:6789/HelloWorld.html

۴. تست خطای 404: http://10.182.63.36:6789/NoSuchFile.html

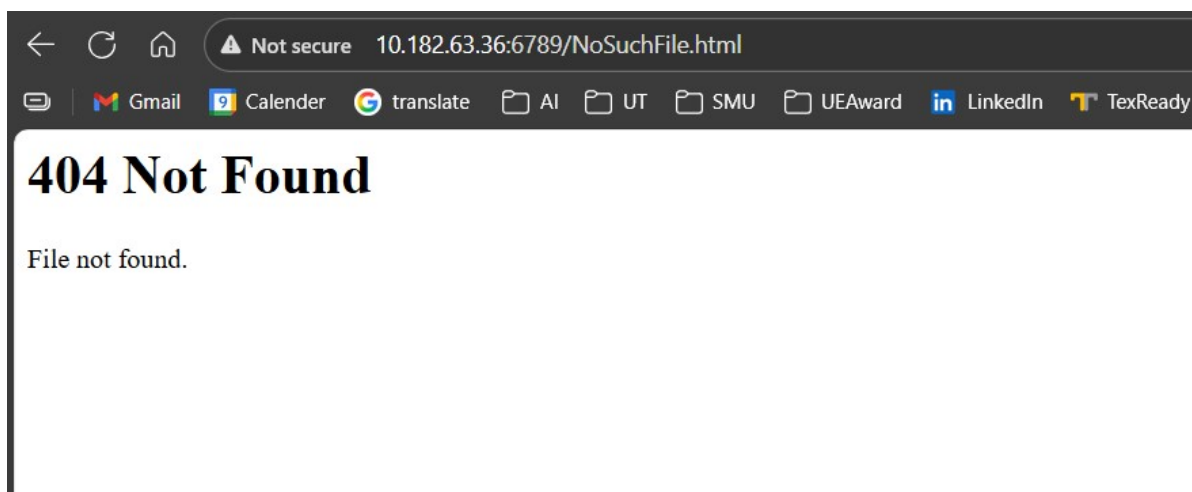
۲ اسکرین شات ها



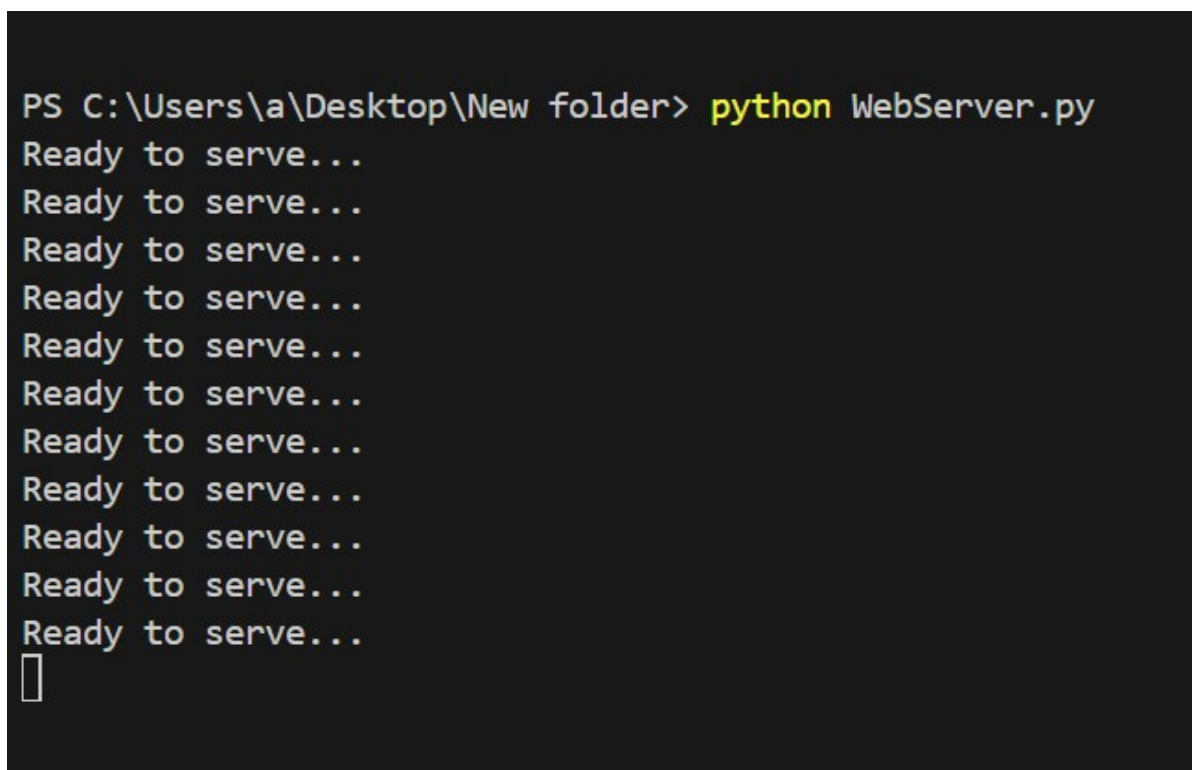
شکل ۱: دریافت موفق HelloWorld.html از طریق IP شبکه محلی (10.182.63.36:6789).



شکل ۲: دریافت موفق HelloWorld.html از طریق localhost با آدرس 127.0.0.1:6789.



شکل ۳: پاسخ صحیح 404 Not Found هنگام درخواست فایل ناموجود.



شکل ۴: ترمینال سرور هنگام گوش دادن روی پورت 6789 و نمایش پیام Ready to serve...

۱.۲ توضیح فنی و تحلیل

- ایجاد و آماده سازی سوکت: ساخت سوکت TCP، bind روی پورت ۶۷۸۹ و listen(1) برای یک اتصال همزمان.
- پذیرش اتصال و دریافت درخواست: با accept() اتصال برقرار و با recv() داده HTTP دریافت می شود.
- تجزیه درخواست: مسیر فایل از خط اول درخواست GET استخراج می شود (عنصر دوم آرایه split).
- پاسخ ۲۰۰: ارسال هدر استاندارد شامل Status-Line، Content-Type و Content-Length و سپس بدنه فایل.
- پاسخ ۴۰۴: در صورت رخداد IOError (عدم وجود فایل)، ارسال بدنه HTML ساده به همراه هدر 404 Not Found.

۲.۲ نحوه تکرار آزمایش

۱. فایل های WebServer.py و HelloWorld.html را در یک پوشه قرار دهید.
۲. سرور را اجرا کنید و از مرورگر به آدرس های بخش قبل بروید.
۳. برای تست خطا، نام فایلی که وجود ندارد را درخواست کنید تا 404 دریافت شود.

مراجع